Battery State-of-Charge & Remaining-Charge-Time Estimation

Project README - detailed view with outputs

Vuda Jaya Sai Ganesh

GROUP S2

1. Project overview

State of charge (SOC) tells a battery-management system how much usable energy is left in a cell, but it cannot be measured directly - it has to be estimated from signals such as voltage, current and temperature. This project builds a frozen, reproducible machine-learning pipeline that estimates:

- SOC of a Li-ion cell, in percent (main target).
- Remaining charging time, in seconds, inside constant-current / constant-voltage (CC-CV) charging segments (secondary target).

The project follows a strict protocol: the data are split once by file (never by row), candidate models are compared with grouped cross-validation on the training files only, the final model is chosen with a pre-agreed rule, and the held-out files are scored exactly once. The final deliverable is a packaged repository with command-line tools, tests, an example workflow, figures, tables, a report and a presentation.

2. Results at a glance (held-out files, scored once)

Target	Frozen model	MAE	RMSE	R2
SOC (%)	Random Forest(tuned)	8.6858	20.4153	-1.3122(%)
Remaining charge time (s)	MLP(ANN)	9001.750	18410.150	-0.160(S)

3. Dataset and frozen split

- Li-ion drive-cycle dataset; features: voltage_v, current_a, cell_temp_c, power_w, ambient_temp_c; target: soc_percent.
- Frozen split: GroupShuffleSplit by file_id (test_size = 0.2, random_state = 42); train/test CSV hashes are stored in the frozen configuration.

Why split by file? Rows from one drive-cycle file are strongly correlated in time. A random row split would put almost identical neighbouring rows in both training and test sets and give over-optimistic scores. Splitting by file_id guarantees that every test file is completely unseen.

4. Methodology and pipeline

1. Load the frozen file-grouped train / held-out split (never re-split).
2. Preprocessing: raw physical features (voltage, current, temperatures, power); no engineered history features.
3. Candidate models: mean predictor, linear regression, random forest, gradient boosting (an MLP was tried in Week 3 and not retained - it memorised the small charging data).
4. Hyper-parameters tuned by grid search on a validation carve-out taken from the training files only (grouped by file).
5. Model selection with grouped cross-validation on the training files: robustness score (MAE, RMSE, worst fold) and the smallest model within 5 % of the best.
6. Freeze the chosen model(s), configuration, split file lists and SHA-256 manifest.
7. Score the held-out files once; run error and failure analysis; package everything with CLI scripts and tests.

4.1 Experiment history

target	method	features	MAE	RMSE	R2	unit
SOC	Mean baseline	-	23.041	27.509	-0.048	%
SOC	Linear Regression	raw	3.473	4.586	0.971	%
SOC	Linear Regression	engineered	23.193	27.771	-0.068	%
SOC	Random Forest	raw	3.284	10.474	0.848	%
SOC	Random Forest	engineered	22.943	28.917	-0.158	%
SOC	Gradient Boosting	raw	3.301	10.082	0.859	%
SOC	Gradient Boosting	engineered	22.599	27.527	-0.049	%
SOC	MLP (ANN)	engineered	25.932	31.611	-0.384	%
Charge time	MLP (ANN)	charging features	9001.750	18410.150	-0.160	s

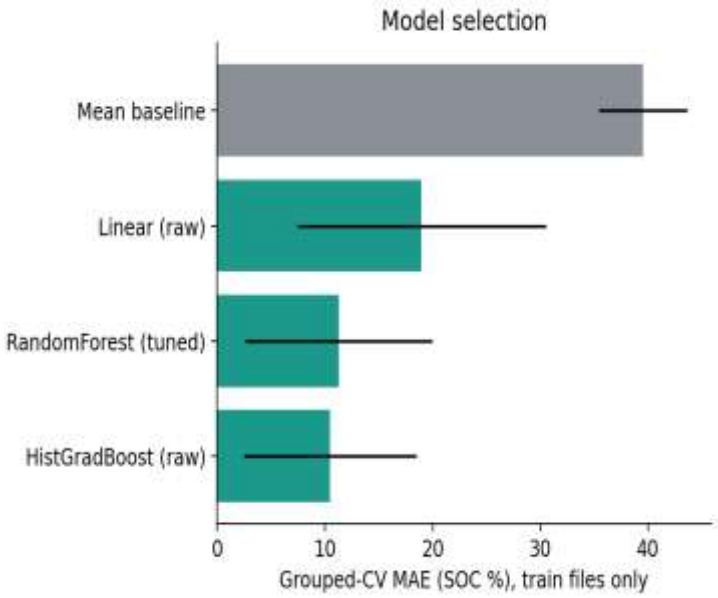
5. Detailed outputs

5.1 Model selection evidence (grouped CV, training files only)

```
===== GROUPED CV (train files only) =====
                      rank   cv_MAE  cv_RMSE   cv_R2  worst_fold_MAE
model
Mean baseline            0  39.5507  41.7687 -8.5913         44.1179
Linear (raw)             1  18.9722  29.1168 -8.7284         29.8468
RandomForest (tuned)     2  11.2925  20.4153 -1.3122         26.3320
HistGradBoost (raw)      3  10.4620  19.3630 -1.3687         24.1248
```

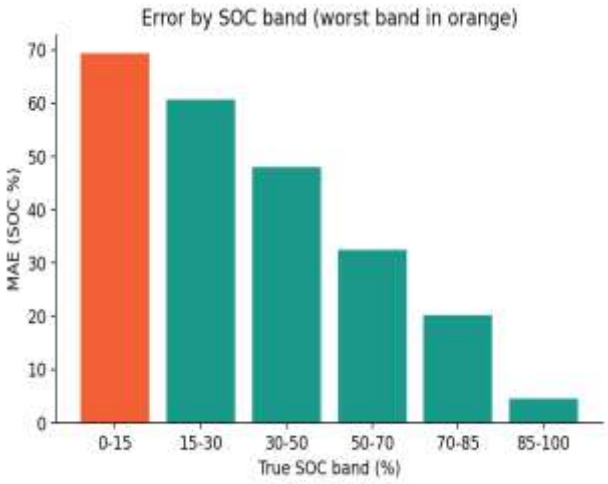
In [4]:

5.2 Final model versus baselines (held-out files)



5.3 Software performance

1	soc_band	MAE	count
2	0-15	69.30485754893358	25
3	15-30	60.747151704336325	36
4	30-50	48.00757813971337	47
5	50-70	32.50482507006922	48
6	70-85	20.254872034596062	36
7	85-100	4.622180321021693	136



5.4 Example input to output workflow

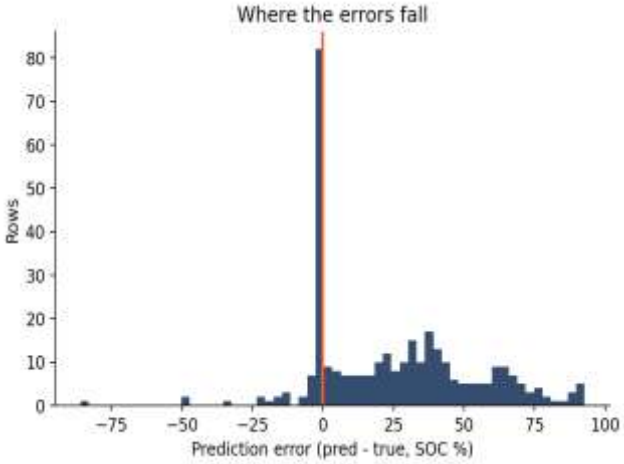
A new user supplies a CSV containing only the raw sensor columns; the CLI returns the same rows with the predicted SOC (and charge time when the needed columns exist).

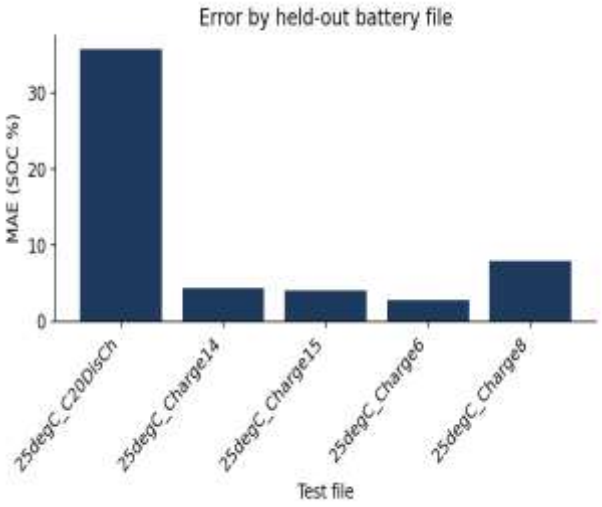
```
python scripts/predict.py --model-dir models --input data/sample_input.csv --output
outputs/sample_output.csv
```

5.5 Automated test results

The suite checks: configuration completeness, disjoint train/test files, valid (finite, 0-100 %) predictions, deterministic predictions, a clear error when a required column is missing, sample metrics matching the recorded values, and a full command-line round trip.

5.6 Figures





6. Error and failure analysis

6.1 Limitations and likely failure conditions

- Small held-out set (few files) - metrics have wide uncertainty.
- Static (history-free) framing of the problem.
- Single dataset / cell type; unseen temperatures, ageing or drive cycles may degrade accuracy.
- Charge-time evidence comes from very few charging files.

7. Repository structure

```
battery_soc_project/
  src/soc_pipeline/   core.py, __init__.py, __main__.py
  scripts/            train.py  evaluate.py  predict.py  build_docs.py  build_readme_docx.py
  models/             frozen .joblib models, frozen_config.json, split_file_ids.json, manifest
  data/               README + sample_input.csv + sample_with_truth.csv
  outputs/            sample_output.csv, metrics.json, test predictions
  figures/ tables/    every figure / table used in the report
  tests/              pytest suite
  docs/               final_report.docx, final_presentation.pptx, this README
```

8. Setup and run instructions

8.1 Local machine

```
git clone <your-repo-url> && cd battery_soc_project
python -m venv .venv && source .venv/bin/activate   # optional
pip install -r requirements.txt
```

```
# 1) predict
python scripts/predict.py  --model-dir models --input data/sample_input.csv --output
outputs/sample_output.csv
# 2) evaluate on held-out files (needs the dataset CSVs, see data/README.md)
python scripts/evaluate.py --model-dir models --test-csv data/dataset_v1_test.csv --full-csv
data/dataset_v1_full.csv --output outputs/metrics.json
# 3) retrain the frozen models (deterministic)
python scripts/train.py    --model-dir models --train-csv data/dataset_v1_train.csv --full-csv
data/dataset_v1_full.csv --out-dir retrained
# 4) tests and documents
pytest -q
python scripts/build_docs.py
python scripts/build_readme_docx.py
```

8.2 Google Colab

1. Run the Day 21 to Day 25 notebooks first so that the frozen artefacts exist in Google Drive (MyDrive/frozen_dataset, day22, day23, day24, day25).
2. Open the Day 26 notebook in Colab and choose Runtime > Run all. Allow Google Drive access when asked.
3. The notebook rebuilds MyDrive/day26/battery_soc_project, runs the tests, generates the report, presentation and this README, and zips the repository.
4. Download this README, final_report.docx and final_presentation.pptx from MyDrive/day26/battery_soc_project/docs/.

9. Reproducibility

- Retraining from frozen_config.json reproduces the stored predictions exactly (checked by tests and by the Day 22 notebook).
- Dataset CSV files are fingerprinted with SHA-256; the split file lists are stored in models/split_file_ids.json.
- Environment versions are recorded in models/frozen_config.json.
- SOC predictions are clipped to 0-100 %; missing input columns raise a clear error.

10. Future scope

- Audit and repair the history-based engineered features that failed to beat raw features.
- Add sequence-aware models (windowed features, GRU / LSTM) and compare them under the same grouped protocol.
- Leave-one-temperature-out and leave-one-drive-cycle-out validation.
- Uncertainty estimates and an out-of-range input detector.
- More charging segments (different rates and temperatures) before treating charge time as production-ready.
- Other cells / chemistries, ageing (state of health) effects, and embedded-hardware tests.

11. Conclusion

A frozen, reproducible pipeline estimates SOC on unseen files; see Section 2 for the measured accuracy once the notebook has been run.